

PRAKTIKUM SIMULASI PERMAINAN UNI

MATA KULIAH IF1221 LOGIKA KOMPUTASIONAL



M. Nabil Rabbani	13525004
Ahmad Humam Isma	13525032
Ghaniyul Amri Caulava	13525106
I Made Adi Kusuma A.	13525121

I. PENJELASAN PERINTAH

I.1 Nama Perintah: startGame

I.1.1 Kegunaan perintah:

Perintah startGame digunakan untuk memulai dan menginisialisasi permainan UNI dari awal. Perintah ini mereset seluruh state permainan sebelumnya (jika ada), meminta jumlah pemain beserta nama masing-masing, membuat dan mengacak deck kartu, membagikan 7 kartu kepada setiap pemain secara acak, menentukan urutan giliran pemain secara acak, serta menginisialisasi discard pile dengan satu kartu angka sebagai kartu awal di meja.

I.1.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Permainan berhasil dimulai

Pengguna mengetik startGame., kemudian memasukkan jumlah pemain (misalnya 3) dan nama masing-masing pemain yang unik (misalnya: Alice, Bob, Carol). Program mengacak urutan pemain, membagikan 7 kartu kepada setiap pemain, menetapkan kartu angka sebagai kartu pertama di discard pile, lalu mengumumkan giliran pemain pertama.

Contoh:

```
?- startGame.  
Masukkan jumlah pemain: 3.  
Masukkan nama pemain 1: Alice.  
Masukkan nama pemain 2: Bob.  
Masukkan nama pemain 3: Carol.  
Urutan pemain: Carol - Alice - Bob.  
Setiap pemain mendapatkan 7 kartu acak.  
Kartu discard top: merah-5.  
Giliran Carol.
```

b. Kondisi Tidak Valid – Jumlah pemain di luar rentang

Jika pengguna memasukkan jumlah pemain kurang dari 2 atau lebih dari 4, program akan menampilkan pesan kesalahan dan meminta input ulang hingga nilai yang valid dimasukkan.

Contoh:

```
Masukkan jumlah pemain: 5.  
Mohon masukkan angka antara 2 - 4.  
Masukkan jumlah pemain: 2.
```

c. Kondisi Tidak Valid – Nama pemain duplikat

Jika pengguna memasukkan nama yang sudah dipakai oleh pemain sebelumnya, program menolak nama tersebut dan meminta input nama baru.

Contoh:

```
Masukkan nama pemain 1: Alice.  
Masukkan nama pemain 2: Alice.  
Nama sudah digunakan. Masukkan nama lain: Bob.
```

I.1.3 Dampak perintah terhadap jalannya permainan (state permainan):

startGame mengubah atau menetapkan seluruh state permainan:

1. urutan_pemain: ditetapkan dengan urutan acak dari daftar nama pemain.
2. giliran: diset ke pemain pertama dalam urutan acak.
3. arah_permainan: diset ke kanan (searah jarum jam).
4. deck: diisi dengan deck kartu lengkap yang telah diacak dan dikurangi kartu yang dibagikan.
5. kartu_pemain: setiap pemain mendapatkan 7 kartu acak.
6. discard_top dan discard_pile/1: diinisialisasi dengan satu kartu angka.
7. warna_aktif: diset sesuai warna kartu awal di discard pile.
8. permainan_aktif: ditambahkan untuk menandai permainan sedang berlangsung.

I.2 Nama Perintah: mainkanKartu

I.2.1 Kegunaan perintah:

Perintah mainkanKartu adalah aksi utama yang digunakan pemain untuk memainkan salah satu kartu dari tangannya berdasarkan nomor urut kartu (dimulai dari 1). Kartu yang dimainkan harus valid, yaitu: memiliki warna yang sama dengan warna aktif, memiliki jenis yang sama dengan kartu teratas di discard pile, atau merupakan kartu wild (berwarna hitam). Setelah kartu dimainkan, efek kartu diterapkan dan giliran berpindah ke pemain berikutnya.

I.2.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Memainkan kartu berwarna sama

Kartu aktif berwarna merah. Pemain memiliki kartu merah-7 di posisi ke-2. Pemain mengetik mainkanKartu(2). dan kartu tersebut berhasil dimainkan.

Contoh:

```
?- mainkanKartu(2).
```

Alice memainkan kartu: merah-7.
Giliran Bob.

b. Kondisi Valid – Memainkan kartu skip

Pemain memainkan kartu skip. Pemain berikutnya kehilangan giliran dan giliran langsung berpindah ke pemain setelahnya.

Contoh:

```
?- mainkanKartu(3).  
Alice memainkan kartu: merah-skip.  
Pemain berikutnya kehilangan giliran.  
Giliran Carol.
```

c. Kondisi Valid – Memainkan kartu wild

Pemain memainkan kartu hitam-wild. Program meminta pemain memilih warna baru, kemudian warna aktif berubah sesuai pilihan.

Contoh:

```
?- mainkanKartu(5).  
Alice memainkan kartu: hitam-wild.  
Pilih warna baru (merah/kuning/hijau/biru) diakhiri  
dengan titik: hijau.  
Warna aktif permainan diubah menjadi hijau.  
Giliran Bob.
```

d. Kondisi Tidak Valid – Nomor kartu di luar jangkauan

Pemain memasukkan nomor urut yang lebih besar dari jumlah kartu di tangannya atau nomor kurang dari 1. Program menampilkan pesan kesalahan.

Contoh:

```
?- mainkanKartu(10).  
Nomor kartu di luar jangkauan kartu di tangan Anda.
```

e. Kondisi Tidak Valid – Kartu tidak cocok

Kartu yang dipilih tidak memiliki warna aktif, jenis yang sama dengan discard top, dan bukan kartu wild. Program menolak dan meminta pemain memilih ulang.

Contoh:

```
?- mainkanKartu(1).
```

Kartu tidak valid! Silakan masukkan pilihan kartu kembali.

f. Kondisi Tidak Valid – Permainan belum dimulai

Jika `startGame` belum dijalankan atau permainan sudah selesai, perintah ini ditolak dengan pesan informasi.

Contoh:

```
?- mainkanKartu(1).  
Permainan belum dimulai atau sudah selesai. Ketik  
startGame. untuk memulai.
```

I.2.3 Dampak perintah terhadap jalannya permainan (state permainan):

`mainkanKartu` mengubah state permainan sebagai berikut:

1. `kartu_pemain`: kartu yang dimainkan dihapus dari tangan pemain aktif.
2. `discard_top`: diperbarui ke kartu yang baru saja dimainkan.
3. `discard_pile`: kartu yang dimainkan ditambahkan ke tumpukan discard.
4. `warna_aktif`: diperbarui sesuai warna kartu yang dimainkan (atau pilihan pemain jika wild).
5. `giliran`: berpindah ke pemain berikutnya (atau melompat 1 untuk `skip/draw_two`).
6. `arah_permainan`: dibalik jika kartu reverse dimainkan.
7. `kartu_pemain` pemain berikutnya: bertambah 2 kartu jika `draw_two`, atau 4 kartu jika `wild_draw_four`.
8. `permainan_aktif`: dihapus jika pemain kehabisan kartu (kondisi menang).

I.3 Nama Perintah: `ambilKartu`

I.3.1 Kegunaan perintah:

Perintah `ambilKartu` adalah aksi utama yang digunakan pemain ketika tidak bisa atau tidak ingin memainkan kartu dari tangannya. Perintah ini mengambil satu kartu teratas dari deck dan menambahkannya ke tangan pemain aktif, kemudian memindahkan giliran ke pemain berikutnya.

I.3.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Mengambil kartu dari deck

Pemain tidak memiliki kartu yang valid untuk dimainkan. Pemain mengetik `ambilKartu`. dan mendapatkan satu kartu baru dari deck.

Contoh:

```
?- ambilKartu.  
Bob mendapatkan kartu: kuning-3.  
Giliran Carol.
```

b. Kondisi Tidak Valid – Permainan belum dimulai

Jika permainan belum dimulai atau telah selesai, perintah ditolak.

Contoh:

```
?- ambilKartu.  
Permainan belum dimulai atau sudah selesai. Ketik  
startGame. untuk memulai.
```

I.3.3 Dampak perintah terhadap jalannya permainan (state permainan):

ambilKartu mengubah state permainan sebagai berikut:

1. deck/1: kartu teratas dihapus dari deck.
2. kartu_pemain/2: satu kartu baru ditambahkan ke tangan pemain aktif.
3. giliran/1: berpindah ke pemain berikutnya.

I.4 Nama Perintah: lihatCommand

I.4.1 Kegunaan perintah:

Perintah lihatCommand adalah aksi pendukung yang menampilkan daftar seluruh perintah yang tersedia dalam permainan, baik aksi utama maupun aksi pendukung, beserta sintaks dan keterangan singkatnya. Perintah ini tidak memengaruhi state permainan dan dapat dipanggil kapan saja, termasuk sebelum permainan dimulai.

I.4.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Menampilkan daftar perintah

Pemain mengetik lihatCommand. dan program menampilkan seluruh daftar perintah yang dapat digunakan.

Contoh:

```
?- lihatCommand.  
Berikut adalah perintah umum yang dapat digunakan  
dalam permainan:  
- startGame.                : Memulai permainan baru.  
- lihatCommand.            : Menampilkan daftar perintah  
ini.
```

- mainkanKartu(Nomor). : Memainkan kartu berdasarkan nomor urut di tangan.
- ambilKartu. : Mengambil satu kartu dari tumpukan (deck).
- lihatKartu. : Melihat daftar kartu yang ada di tangan Anda saat ini.
- cekInfo. : Melihat informasi permainan.
- uni(Nomor). : Memanggil "UNI" saat memainkan kartu kedua terakhir.
- tangkap>NamaPemain). : Menangkap pemain yang lupa memanggil "UNI".
- saveGame. : Menyimpan progres permainan saat ini.
- loadGame. : Memuat progres permainan yang disimpan.

I.4.3 Dampak perintah terhadap jalannya permainan (state permainan):

Perintah lihatCommand bersifat read-only dan tidak mengubah state permainan apapun. Tidak ada fakta dinamis yang ditambahkan, diubah, maupun dihapus saat perintah ini dipanggil.

I.5 Nama Perintah: lihatKartu

I.5.1 Kegunaan perintah:

Perintah lihatKartu adalah aksi pendukung yang digunakan untuk menampilkan daftar kartu yang sedang dipegang oleh pemain yang giliran bermainnya saat ini, beserta nomor urut masing-masing kartu. Informasi ini membantu pemain menentukan kartu mana yang akan dimainkan. Perintah ini hanya dapat digunakan ketika permainan sedang aktif

I.5.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Melihat kartu saat giliran aktif

Pemain aktif mengetik lihatKartu. dan program menampilkan seluruh kartu di tangannya beserta nomor urutnya.

Contoh:

```
?- lihatKartu.
Kartu di tangan Alice:
1. merah-3
2. kuning-skip
```

3. hijau-7
4. hitam-wild
5. biru-2
6. merah-reverse
7. kuning-5

b. Kondisi Tidak Valid – Permainan belum dimulai

Jika permainan belum dimulai, program menampilkan pesan peringatan.

Contoh:

```
?- lihatKartu.  
Permainan belum dimulai!
```

I.5.3 Dampak perintah terhadap jalannya permainan (state permainan):

Perintah `lihatKartu` bersifat read-only dan tidak mengubah state permainan apapun. Tidak ada fakta dinamis yang ditambahkan, diubah, maupun dihapus saat perintah ini dipanggil.

I.6 Nama Perintah: cekInfo

I.6.1 Kegunaan perintah:

Perintah `cekInfo` adalah aksi pendukung yang menampilkan ringkasan informasi dan status permainan saat ini. Informasi yang ditampilkan meliputi: nama pemain yang sedang mendapat giliran, kartu teratas pada discard pile, warna aktif permainan, serta nama dan jumlah kartu yang dipegang oleh setiap pemain. Perintah ini membantu semua pemain memantau kondisi permainan tanpa mempengaruhi jalannya permainan.

I.6.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Melihat info saat permainan berjalan

Saat permainan sedang berlangsung, pemain mengetik `cekInfo`. dan program menampilkan seluruh informasi status permainan saat ini.

Contoh:

```
?- cekInfo.  
Giliran saat ini: Alice  
Kartu teratas: merah-5  
Warna aktif: merah  
Jumlah kartu pemain:  
- Alice: 7 kartu  
- Bob: 6 kartu
```


- Carol: 8 kartu

b. Kondisi Tidak Valid – Permainan belum dimulai

Jika permainan belum dimulai atau telah selesai, program menampilkan pesan peringatan.

Contoh:

```
?- cekInfo.  
Permainan belum dimulai!
```

I.6.3 Dampak perintah terhadap jalannya permainan (state permainan):

Perintah `cekInfo` bersifat read-only dan tidak mengubah state permainan apapun. Tidak ada fakta dinamis yang ditambahkan, diubah, maupun dihapus saat perintah ini dipanggil.

I.7 Nama Perintah: uni

I.7.1 Kegunaan perintah:

Perintah `uni` adalah aksi utama yang digunakan pemain untuk memainkan kartu kedua terakhirnya sekaligus menyerukan "UNI". Perintah ini hanya valid ketika pemain memiliki tepat 2 kartu di tangan. Jika berhasil, pemain tidak akan ditandai sebagai 'lupa UNI' sehingga aman dari perintah tangkap. Kartu yang dipilih tetap harus memenuhi aturan validasi (warna cocok, jenis cocok, atau wild).

I.7.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Memainkan kartu kedua terakhir dengan seruan UNI

Pemain memiliki tepat 2 kartu. Ia mengetik `uni(1)` untuk memainkan kartu pertama di tangannya. Kartu tersebut valid, sehingga program mencetak seruan UNI dan memproses kartu seperti biasa.

Contoh:

```
?- uni(1).  
Bob menyerukan UNI!  
Bob memainkan kartu: kuning-5.  
Giliran Carol.
```

b. Kondisi Tidak Valid – Pemain memiliki lebih atau kurang dari 2 kartu

Jika pemain memiliki jumlah kartu bukan 2 (misalnya 5 kartu), perintah `uni` ditolak karena tidak memenuhi syarat UNI.

Contoh:

```
?- uni(2).
```

Perintah tidak valid. Kartu Anda tidak akan tersisa satu setelah aksi ini.

c. Kondisi Tidak Valid – Kartu yang dipilih tidak valid

Pemain memiliki 2 kartu namun kartu yang dipilih tidak cocok dengan warna aktif maupun jenis kartu di discard pile, dan bukan wild.

Contoh:

```
?- uni(2).
```

Bob menyerukan UNI!

Kartu tidak valid! Silahkan masukkan pilihan kartu kembali.

I.7.3 Dampak perintah terhadap jalannya permainan (state permainan):

uni mengubah state permainan sebagai berikut:

1. kartu_pemain: kartu yang dimainkan dihapus dari tangan pemain aktif (sisa 1 kartu).
2. discard_top dan discard_pile: diperbarui dengan kartu yang baru dimainkan.
3. warna_aktif: diperbarui sesuai warna kartu (atau pilihan pemain jika wild).
4. lupa_uni: TIDAK ditambahkan untuk pemain ini (aman dari tangkap).
5. giliran: berpindah ke pemain berikutnya sesuai efek kartu.
6. permainan_aktif: dihapus jika sisa kartu = 0 (endGame dipanggil).

I.8 Nama Perintah: tangkap

I.8.1 Kegunaan perintah:

Perintah tangkap adalah aksi pendukung yang dapat dipanggil oleh pemain mana pun untuk menghukum pemain lain yang lupa menyerukan "UNI" saat memainkan kartu kedua terakhirnya menggunakan mainkanKartu (bukan uni). Jika tuduhan benar, pemain yang tertangkap mendapat 2 kartu penalti. Jika tuduhan salah, pemain yang menuduh mendapat 1 kartu penalti.

I.8.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Tuduhan benar (target lupa UNI)

Bob menggunakan mainanKartu saat punya 2 kartu tanpa menyerukan UNI. Alice mendeteksi ini dan mengetik tangkap(Bob). Bob terbukti lupa UNI dan mendapat 2 kartu penalti.

Contoh:

```
?- tangkap(Bob) .  
Bob tertangkap tidak menyerukan UNI!  
Bob mendapat 2 kartu penalti.
```

b. Kondisi Valid – Tuduhan salah (target tidak lupa UNI)

Alice menuduh Carol, namun Carol sudah menyerukan UNI dengan benar sebelumnya. Alice sebagai penuduh mendapat 1 kartu penalti.

Contoh:

```
?- tangkap(Carol) .  
Tuduhan tidak valid! Target memiliki lebih dari 1 kartu  
atau sudah menyerukan UNI.  
Alice mendapat 1 kartu penalti secara acak.
```

c. Kondisi Tidak Valid – Menangkap diri sendiri

Pemain mencoba menangkap dirinya sendiri.

Contoh:

```
?- tangkap(Alice) .  
Kocak! Kamu tidak bisa menangkap dirimu sendiri..
```

I.8.3 Dampak perintah terhadap jalannya permainan (state permainan):

tangkap mengubah state permainan sebagai berikut:

1. kartu_pemain target: bertambah 2 kartu jika tuduhan benar.
2. kartu_pemain penuduh: bertambah 1 kartu jika tuduhan salah.
3. lupa_uni: dihapus untuk target jika terbukti (sudah tertangkap).
4. deck: berkurang sejumlah kartu penalti yang diambil.
5. giliran: tidak berubah (tangkap adalah aksi pendukung).

I.9 Nama Perintah: tantang

I.9.1 Kegunaan perintah:

Perintah tantang adalah aksi utama yang dapat digunakan pemain berikutnya untuk menantang keabsahan penggunaan kartu wild draw four. Kartu wild draw four hanya boleh dimainkan jika pemain tidak memiliki kartu lain yang cocok dengan warna aktif sebelumnya. Jika penantang menang (terbukti pemain WD4 masih punya kartu

cocok), pemain yang memainkan WD4 mendapat 4 kartu penalti. Jika penantang kalah, penantang mendapat 6 kartu penalti.

I.9.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Tantangan berhasil (WD4 player curang)

Carol memainkan wild draw four padahal masih punya kartu merah. Bob (pemain berikutnya yang seharusnya kena efek) memilih untuk menantang. Tangan Carol diungkap dan terbukti ada kartu merah. Carol mendapat 4 penalti dan Bob tidak jadi mengambil 4 kartu.

Contoh:

```
?- tantang.  
Tantangan dilakukan.  
Memeriksa kartu Carol ...  
Tantangan berhasil! Carol mendapatkan 4 kartu acak.
```

b. Kondisi Valid – Tantangan gagal (WD4 player jujur)

Carol memainkan wild draw four dan memang tidak punya kartu merah. Bob menantang namun terbukti Carol jujur. Bob mendapat 6 kartu penalti.

Contoh:

```
?- tantang.  
Tantangan dilakukan.  
Memeriksa kartu Carol ...  
Tantangan gagal. Bob mendapatkan 6 kartu acak.
```

c. Kondisi Tidak Valid – Tidak ada WD4 yang dapat ditantang

Pemain mengetik tantang. namun kartu yang terakhir dimainkan bukan wild draw four.

Contoh:

```
?- tantang.  
Tidak ada kartu wild_draw_four yang bisa ditantang saat ini..
```

I.9.3 Dampak perintah terhadap jalannya permainan (state permainan):

tantang mengubah state permainan sebagai berikut:

Jika tantangan berhasil:

1. kartu_pemain Tertantang: bertambah 4 kartu.
2. status_tantang: dihapus.

3. giliran: tidak berubah (Penantang tetap giliran dan bebas memilih aksi).

Jika tantangan gagal:

1. kartu_pemain Penantang: bertambah 6 kartu.
2. status_tantang: dihapus.
3. giliran: berpindah ke pemain berikutnya (ganti_giliran(1)).

I.10 Nama Perintah: endGame

I.10.1 Kegunaan perintah:

Perintah endGame dipanggil secara otomatis dari mainkanKartu ketika seorang pemain menghabiskan seluruh kartunya (sisanya = 0). Perintah ini mengakhiri sesi permainan, menghitung total nilai poin dari sisa kartu setiap pemain, lalu menampilkan peringkat akhir dari skor terkecil (terbaik) ke terbesar. Permainan tidak dapat dilanjutkan setelah endGame dipanggil karena fakta permainan_aktif dihapus.

I.10.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Pemain menghabiskan kartu terakhirnya

alice memainkan kartu terakhir. Program otomatis memanggil endGame, menampilkan rincian poin, dan mencetak peringkat akhir.

Contoh:

```
alice memainkan kartu: merah-7.  
Permainan selesai! alice menghabiskan semua kartunya!  
Berikut perhitungan poin sisa kartu.  
alice: kartu habis = 0 poin  
bob: kuning-skip + biru-3 = 10 + 3 = 13 poin  
carol: hijau-2 + merah-9 + hitam-wild = 2 + 9 + 20 = 31 poin
```

```
Urutan pemenang:  
1. alice (0 poin)  
2. bob (13 poin)  
3. carol (31 poin)
```

Selamat, alice menjadi pemenang!

I.10.3 Dampak perintah terhadap jalannya permainan (state permainan):

endGame merubah state permainan sebagai berikut:

1. permainan_aktif: di-retract, menandai permainan telah selesai.
2. Fakta lain (kartu_pemain, urutan_pemain, dll.) tidak diubah sehingga skor tetap dapat dilihat.

I.11 Nama Perintah: saveGame

I.11.1 Kegunaan perintah:

Perintah `saveGame` menyimpan state permainan yang sedang berjalan ke file `.txt`. Program meminta nama file dari pengguna (tanpa ekstensi) lalu menulis data menggunakan `tell/told`. Komponen yang disimpan: `urutan_pemain`, `giliran`, `discard_top`, `warna_aktif`, `arah_permainan`, `status_uni` setiap pemain, dan kartu di tangan setiap pemain. Format penyimpanan menggunakan pemisah titik dua (`:`). `saveGame` tidak dapat dilakukan jika sedang ada `status_tantang` (pemain sedang diharuskan memilih `tantang/ambilKartu`). Deck tidak disimpan; saat `loadGame` dipanggil, deck baru akan dibuat secara acak.

I.11.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – Permainan aktif berhasil disimpan

Permainan sedang berjalan. Pengguna memasukkan nama file dan state tersimpan.

Contoh:

```
?- saveGame.  
Masukkan nama file penyimpanan: saveku.  
Status permainan berhasil disimpan ke saveku.txt.
```

b. Kondisi Tidak Valid – Ada `status_tantang` aktif

Tidak bisa menyimpan saat pemain wajib memilih `tantang` atau `ambilKartu`.

Contoh:

```
?- saveGame.  
Command ini tidak dapat digunakan ketika pemain  
diharuskan memilih  
command tertentu (misal: tantang/ambilKartu).
```

I.11.3 Dampak perintah terhadap jalannya permainan (state permainan):

`saveGame` tidak mengubah state permainan di memori sama sekali (pure read).

I.12 Nama Perintah: `loadGame`

I.12.1 Kegunaan perintah:

Perintah loadGame memuat state permainan dari file .txt yang sebelumnya disimpan oleh saveGame. Program meminta nama file (tanpa ekstensi) dari pengguna. Jika file ditemukan, reset_game_state dipanggil untuk membersihkan state saat ini, kemudian setiap baris dibaca dan diproses oleh proses_baris/1 untuk me-restore urutan_pemain, giliran, discard_top, warna_aktif, arah_permainan, status_uni, dan kartu per pemain. Setelah itu, restore_deck_and_discard dipanggil untuk membuat deck baru secara acak (deck tidak disimpan dalam file).

I.12.2 Skenario-skenario beserta contoh penggunaannya:

File saveku.txt ada. Program memuat state dan melanjutkan dari giliran terakhir.

Contoh:

```
?- loadGame.  
Masukkan nama file yang akan dimuat: saveku.  
Status permainan berhasil dimuat dari saveku.txt.  
Melanjutkan giliran bob.
```

a. Kondisi Tidak Valid – File tidak ditemukan

Nama file yang dimasukkan tidak ada di direktori kerja.

Contoh:

```
?- loadGame.  
Masukkan nama file yang akan dimuat: tidakada.  
File tidak ditemukan.
```

I.12.3 Dampak perintah terhadap jalannya permainan (state permainan):

loadGame merubah state permainan sebagai berikut:

1. Seluruh fakta dinamis dihapus terlebih dahulu via reset_game_state.
2. urutan_pemain, giliran, discard_top, warna_aktif, arah_permainan,
3. status_uni, kartu_pemain: di-restore dari file.
4. deck: diisi dengan deck BARU yang diacak (bukan deck yang tersimpan).
5. discard_pile: diinisialisasi ulang dari discard_top yang dimuat.
6. permainan_aktif: di-assert kembali setelah semua fakta berhasil dimuat.

I.13 Nama Perintah: godsHand

I.13.1 Kegunaan perintah:

Perintah `godsHand` adalah fitur yang memindahkan satu kartu secara acak dari satu pemain (donor) ke pemain lain (penerima) yang dipilih juga secara acak. Perintah ini aktif ketika dipanggil oleh pemain yang sedang mendapat giliran. Terdapat satu pengecualian: jika semua pemain memiliki tepat 1 kartu, `godsHand` tidak diaktifkan untuk menjaga keseimbangan permainan. Donor dan penerima dijamin berbeda pemain. Giliran tidak berubah setelah `godsHand`.

I.13.2 Skenario-skenario beserta contoh penggunaannya:

a. Kondisi Valid – `godsHand` berhasil memindahkan kartu

Tidak semua pemain punya 1 kartu. Program memilih donor dan penerima secara acak.

Contoh:

```
?- godsHand.  
Tuhan telah berkehendak.  
Kartu merah-5 milik carol berpindah ke tangan bob!  
Giliran alice.
```

b. Kondisi Tidak Valid – Semua pemain memiliki tepat 1 kartu

Kondisi keseimbangan: tidak ada kartu yang dipindahkan.

Contoh:

```
?- godsHand.  
Keseimbangan terjaga. Semua pemain memiliki 1 kartu.  
God's Hand tidak aktif.
```

I.13.3 Dampak perintah terhadap jalannya permainan (state permainan):

`godsHand` merubah state permainan sebagai berikut:

1. `kartu_pemain donor`: berkurang 1 kartu (kartu yang dipindahkan dihapus).
2. `kartu_pemain penerima`: bertambah 1 kartu (kartu donor ditambahkan di awal list tangan).
3. `giliran`: tidak berubah.
4. Seluruh pemilihan donor, kartu, dan penerima dilakukan secara acak menggunakan `random`.

II. HASIL EKSEKUSI PROGRAM

1. startGame

```
| ?- startGame.  
Masukkan jumlah pemain: 4  
.  
  
Masukkan nama pemain 1: 'Wahyu'.  
Masukkan nama pemain 2: 'Brata'.  
Masukkan nama pemain 3: 'Danta'.  
Masukkan nama pemain 4: 'Made'.  
  
Urutan pemain: Wahyu - Danta - Brata - Made.  
  
Setiap pemain mendapatkan 7 kartu acak.  
  
Kartu discard top: biru-0.  
  
Giliran Wahyu.  
  
true ? |
```

Memulai permainan dengan predikat 'startGame' dengan input 4 pemain, dengan nama pemain tertera pada gambar.

2. lihatCommand

```
| ?- lihatCommand.  
Berikut adalah perintah umum yang dapat digunakan dalam permainan:  
- startGame.           : Memulai permainan baru.  
- lihatCommand.        : Menampilkan daftar perintah ini.  
- mainkanKartu(Nomor). : Memainkan kartu berdasarkan nomor urut di tangan.  
- ambilKartu.          : Mengambil satu kartu dari tumpukan (deck).  
- lihatKartu.          : Melihat daftar kartu yang ada di tangan Anda saat ini.  
- cekInfo.             : Melihat informasi permainan (giliran, arah, warna aktif, kartu teratas).  
- uni(Nomor).          : Memanggil "UNI" bersamaan dengan memainkan kartu kedua terakhir (tersisa 1 kartu).  
- tangkap>NamaPemain). : Menangkap pemain yang lupa memanggil "UNI".  
- saveGame.            : Menyimpan progres permainan saat ini.  
- loadGame.            : Memuat progres permainan yang sebelumnya disimpan.  
  
(31 ms) yes  
| ?- |
```

Menampilkan command command yang tersedia dalam program permainan 'UNI' melalui predikat 'lihatCommand'.

3. lihatKartu

```
| ?- lihatKartu.  
Kartu di tangan Wahyu:  
1. biru-5  
2. hijau-8  
3. hijau-0  
4. hitam-wild  
5. biru-skip  
6. kuning-reverse  
7. merah-9
```

Melihat kartu giliran pemain sekarang dengan menggunakan predikat 'lihatKartu'.

4. mainkanKartu

- a. Kasus tidak valid

```
| ?- mainkanKartu(2).  
Kartu tidak valid! Silakan masukkan pilihan kartu kembali.
```

Output tidak valid dikarenakan kartu yang dimainkan tidak sesuai dengan kartu yang sedang ada di permainan saat ini. Dalam kasus ini kartu yang ada di permainan saat ini adalah **biru-0**, sedangkan kartu yang dimainkan adalah **hijau-8**.

- b. Kasus valid

```
| ?- mainkanKartu(3).  
Wahyu memainkan kartu: hijau-0.  
Giliran Danta.
```

Output valid karena kartu yang dimainkan sesuai dengan kartu yang sedang ada di permainan saat ini. Dalam kasus ini, kartu yang ada di permainan saat ini adalah **biru-0**, kartu yang dimainkan adalah **hijau-0**. Valid karena sesuai dengan aturan dimana kartu yang dimainkan harus memiliki angka yang sama atau warna yang sama atau kartu wild.

- Kartu Skip

```
| ?- mainkanKartu(4).
Wahyu memainkan kartu: biru-skip.
Pemain berikutnya kehilangan giliran.
Giliran Brata.
```

Output dari mainkanKartu skip yakni pemain berikutnya akan kehilangan giliran dan lanjut ke pemain selanjutnya. Misal giliran saat ini adalah 'Made', 'Wahyu', 'Danta', 'Brata'. Wahyu memainkan kartu skip maka 'Danta' akan kehilangan giliran permainan dan lanjut ke giliran 'Brata'.

- Kartu Reverse

```
(63 ms) yes
| ?- mainkanKartu(10).
Brata memainkan kartu: biru-reverse.
Arah permainan berputar.
Giliran Danta.
```

Output dari mainkanKartu reverse yakni giliran berikutnya akan berbalik. Misal giliran saat ini adalah 'Made', 'Wahyu', 'Danta', 'Brata'. Brata memainkan kartu reverse, dimana giliran yang seharusnya adalah 'Made', tetapi karena efek dari kartu reverse, maka giliran yang sekarang kembali ke 'Danta' dan berlanjut ke Wahyu dan seterusnya.

- Kartu Draw Two

```
| ?- mainkanKartu(1).
Danta memainkan kartu: biru-draw_two.
Pemain berikutnya mengambil 2 kartu dan kehilangan giliran.
Giliran Made.
```

Disini 'Danta' memainkan kartu **biru-draw_two**, dimana efeknya adalah pemain berikutnya yakni 'Wahyu' mendapatkan 2 kartu acak dan kehilangan giliran, dan giliran berlanjut ke 'Made'.

5. cekInfo

```
| ?- cekInfo.  
Giliran saat ini: Danta  
Kartu teratas: hijau-0  
Warna aktif: hijau  
Jumlah kartu pemain:  
- Wahyu: 6 kartu  
- Danta: 7 kartu  
- Brata: 7 kartu  
- Made: 7 kartu
```

Dengan menggunakan predikat ‘cekInfo’ pemain dapat melihat informasi permainan yang sedang berlangsung seperti giliran, kartu teratas, warna aktif, dan jumlah kartu dari setiap pemain.

6. tantang

a. Tantang berhasil

```
| ?- mainkanKartu(3).  
Danta memainkan kartu: hijau-9.  
Giliran Brata.
```

```
| ?- lihatKartu.  
Kartu di tangan Brata:  
1. merah-8  
2. kuning-1  
3. hitam-wild_draw_four  
4. hijau-7  
5. merah-7  
6. merah-1  
7. merah-2
```

```
| ?- mainkanKartu(3).  
Brata memainkan kartu: hitam-wild_draw_four.  
Pilih warna baru (merah/kuning/hijau/biru) diakhiri dengan titik: biru.  
Warna aktif permainan diubah menjadi biru.  
Pemain berikutnya dapat melakukan perintah tantang. atau ambilKartu.  
Giliran Made.
```

```
| ?- tantang.  
Tantangan dilakukan!  
Memeriksa kartu Brata...  
Tantangan berhasil!  
Brata mendapatkan 4 kartu acak.  
Silakan mainkan kartu Anda atau ketik ambilKartu.
```

Kartu teratas saat ini adalah **hijau-9** tetapi ‘Brata’ memainkan kartu **hitam-wild_draw_four**, pemain selanjutnya yaitu ‘Made’ dapat melakukan

command ‘tantang’ dan berhasil karena ‘Brata’ masih mempunyai kartu valid yang bisa dimainkan yakni **hijau-7**. Alhasil tantang berhasil dan Brata mendapatkan kartu acak.

b. Tantang gagal

```
| ?- cekInfo.  
Giliran saat ini: Danta  
Kartu teratas: biru-9  
Warna aktif: biru  
Jumlah kartu pemain:  
- Wahyu: 6 kartu  
- Danta: 6 kartu  
- Brata: 11 kartu  
- Made: 4 kartu  
  
| ?- lihatKartu.  
Kartu di tangan Danta:  
1. hitam-wild_draw_four  
2. merah-skip  
3. kuning-6  
4. merah-0  
5. merah-5  
6. hijau-2  
  
| ?- mainkanKartu(1).  
Danta memainkan kartu: hitam-wild_draw_four.  
Pilih warna baru (merah/kuning/hijau/biru) diakhiri dengan titik: merah.  
Warna aktif permainan diubah menjadi merah.  
Pemain berikutnya dapat melakukan perintah tantang. atau ambilKartu.  
Giliran Wahyu.  
  
| ?- tantang.  
Tantangan dilakukan!  
Memeriksa kartu Danta...  
Tantangan gagal. Wahyu mendapatkan 6 kartu acak.  
Giliran Made.
```

Kartu teratas saat ini adalah **biru-9** dan ‘Danta’ memainkan kartu **hitam-wild_draw_four**. Pemain selanjutnya yakni ‘Wahyu’ dapat melakukan ‘tantang’, tetapi karena ‘Danta’ tidak memiliki kartu yang valid selain kartu wild maka tantang gagal dilakukan dan ‘Wahyu’ mendapatkan 6 kartu acak.

7. ambilKartu

```

(47 ms) yes
| ?- mainkanKartu(2).
Made memainkan kartu: biru-8.
Giliran Brata.

| ?- lihatKartu.
Kartu di tangan Brata:
1. merah-8
2. kuning-1
3. hijau-7
4. merah-7
5. merah-1
6. merah-2
7. hijau-1
8. merah-4
9. merah-draw_two

| ?- ambilKartu.
Brata mendapatkan kartu: kuning-8.
Giliran Danta.

```

‘Brata’ menggunakan predikat ‘ambilKartu’ dikarenakan tidak memiliki kartu valid yang dimainkan karena kartu teratas saat ini adalah **biru-8**, sedangkan ‘Brata’ tidak memiliki kartu berwarna biru maupun kartu berangka 8.

8. tangkap

a. Tangkap berhasil

```

| ?- lihatKartu.
Kartu di tangan Made:
1. merah-reverse
2. hijau-9

yes
| ?- mainkanKartu(2).
Made memainkan kartu: hijau-9.
Giliran Wahyu.

| ?- cekInfo.
Giliran saat ini: Wahyu
Kartu teratas: hijau-9
Warna aktif: hijau
Jumlah kartu pemain:
- Wahyu: 9 kartu
- Danta: 3 kartu
- Brata: 8 kartu
- Made: 1 kartu

```

```
| ?- tangkap('Made').  
Made tertangkap tidak menyerukan UNI.  
Made mendapatkan 2 kartu penalti.  
Giliran Wahyu.
```

Pemain dapat menggunakan predikat ‘tangkap’ ketika mengetahui bahwa pemain lainnya tidak menyuarakan uni. Contohnya ada pada gambar diatas, ‘Made’ tidak menggunakan ‘uni’ ketika hanya tersisa 2 kartu terakhir, sehingga tangkap berhasil dan pemain target mendapatkan 2 kartu penalti.

b. Tangkap gagal

```
lihatKartu.  
Kartu di tangan Danta:  
1. biru-skip  
2. biru-5  
  
| ?- uni(1).  
Danta menyerukan UNI!  
Danta memainkan kartu: biru-skip.  
Pemain berikutnya kehilangan giliran.  
Giliran Made.  
  
| ?- tangkap('Danta').  
Tuduhan tidak valid! Target memiliki lebih dari 1 kartu atau sudah menyerukan UNI.  
Made mendapatkan 1 kartu penalti secara acak.  
Giliran Made.
```

Jika seorang pemain melakukan ‘tangkap’ pada target, tetapi target sudah menyuarakan ‘uni” atau masih memiliki lebih dari 1 kartu, maka ‘tangkap’ gagal dan pemain yang melakukannya akan mendapatkan 1 kartu penalti.

9. uni

```
| ?- uni(1).  
Danta menyerukan UNI!  
Danta memainkan kartu: biru-1.  
Giliran Wahyu.
```

Apabila kartu seorang pemain tersisa 2, maka pemain dapat menggunakan predikat ‘uni’. Dengan menggunakan ‘uni’ pemain akan mengeluarkan kartu valid dan menyerukan ‘UNI!’ supaya tidak terkena command ‘tangkap’ oleh pemain lainnya.

10. saveGame

```
| ?- saveGame.  
Masukkan nama file penyimpanan: gamesatu.  
Status permainan berhasil disimpan ke gamesatu.txt.
```

Dengan menggunakan predikat 'saveGame' pemain dapat menyimpan proses game ke dalam file .txt. Misal pada contoh, proses game disimpan ke dalam file 'gamesatu.txt'.

11. loadGame

```
| ?- loadGame.  
Masukkan nama file yang akan dimuat: gamesatu.  
Status permainan berhasil dimuat dari gamesatu.txt.  
Melanjutkan giliran Made.
```

Dengan menggunakan predikat 'loadGame' pemain dapat melanjutkan game yang disimpan melalui 'saveGame' sebelumnya dengan melakukan input nama file seperti contoh pada gambar.

12. endGame (Permainan Berakhir)

```
| ?- lihatKartu.  
Kartu di tangan Danta:  
1. biru-5
```

```
| ?- mainkanKartu(1).
```

Danta memainkan kartu: biru-5.

Permainan selesai! Danta menghabiskan semua kartunya!

Berikut perhitungan poin sisa kartu.

Wahyu: kuning-draw_two + hitam-wild_draw_four + hijau-6 +
kuning-reverse + merah-9 + kuning-6 + merah-1 + hijau-2 +
merah-5 + merah-reverse + merah-3 = 10 + 20 + 6 + 10 + 9 + 6 +
1 + 2 + 5 + 10 + 3 = 82 poin

Danta: kartu habis = 0 poin

Brata: merah-8 + hijau-7 + merah-1 + merah-2 + merah-draw_two +
merah-6 + hijau-2 = 8 + 7 + 1 + 2 + 10 + 6 + 2 = 36 poin

Made: merah-reverse + kuning-0 + kuning-1 = 10 + 0 + 1 = 11

poin

Urutan pemenang:

1. Danta (0 poin)
2. Made (11 poin)
3. Brata (36 poin)
4. Wahyu (82 poin)

Selamat, Danta menjadi pemenang!

Permainan berakhir ketika ada pemain pertama yang kartunya habis. Contoh pada gambar yang menang adalah 'Danta'. Setelah itu program akan menghitung poin dari pemain lainnya sesuai dengan aturan yang sudah dijelaskan sebelumnya dan permainan pun berakhir.

III. PEMBAGIAN DAN PRESENTASE KERJA

Nama Anggota	NIM	Deskripsi Tugas/Bagian	Persentase (%)
Ghaniyul Amri Caulava	13525106	- startGame - tangkap - tantang - godsHand - Laporan Milestone 3	25 %
Ahmad Humam Isma	13525032	- lihatCommand - endGame - laporan - Laporan Milestone 3	25 %
I Made Adi Kusuma Ardana	13525121	- ambilKartu - mainkanKartu - Laporan Milestone 1 - Laporan Milestone 2	25 %

Nama Anggota	NIM	Deskripsi Tugas/Bagian	Persentase (%)
		- Laporan Milestone 3	
Nabil Rabbani	13525004	- uni - lihatKartu - cekInfo	25 %

IV. LAMPIRAN FORM PENGGUNAAN GENERATIVE AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Ghaniyul Amri Caulava/13525106
Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF 1221
Nama Mata Kuliah : Logika Komputasional
Judul Tugas/Proposal : Praktikum Simulasi Permainan UNI Mata Kuliah IF1221 Logika Komputasional

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya /Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya /Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/ Tidak *	75 %
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya /Tidak*	
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Sumedang, 30 Mei 2026



Ghaniyul Amri Caulava
13525106